ENGINEERING HANDBOOK

Sizzle — Restaurant OS
how the project is built and how it works

A single-tenant demo of a full restaurant floor: guest QR ordering, a kitchen pass, a waiter floor, a counter with a cash drawer, bookings, inventory, hiring and owner reports — one TypeScript codebase, one API, one browser client, realtime throughout.

This document is a map of the code as it stands: the folder tree, the request path, the data model with every collection and index, the state machines, the permission grid, the realtime contract, the build and deploy chain, and the rules that were chosen deliberately (and why). Read it before changing anything that another screen depends on.

Repository `github.com/aniruddhasali800-pixel/ai` · branch `main`
Tenant in the seed: **Saffron & Smoke**, Koregaon Park, Pune (`saffron-and-smoke`)
Live: `https://smart.restaurant.aniruddhasali.in` · API behind it: `https://ai-1-hsus.onrender.com`
Generated 4 October 2026, after commit `bbfa7bb`

01 Contents

02 What the product does · one service period

03 Languages, runtimes and libraries · and why each one

04 System design · topology, request path, realtime

05 Repository layout · annotated tree

06 Backend anatomy · boot, middleware, modules, services

07 HTTP surface · every route and its guard

08 Data management · 19 collections, indexes, counters, TTL

09 State machines · order, table, session, booking

10 The two floor rules · one fire, one label

11 Money · pricing, bills, UPI, payments

12 Auth, sessions and permissions

13 Realtime contract · rooms and event names

14 Frontend anatomy · entries, routes, state, UI system

15 PWA, QR codes and installables

16 Configuration reference

17 Build, test and deploy

18 Runbook · symptoms to files

19 Deliberate mocks, limits and debts

20 Glossary

02 What the product does

Sizzle is the operating system for one restaurant floor. Six kinds of person touch it, and the point of the build is that they are all looking at the same rows in the same database at the same second.

Who	Where they live	What they can do
Guest at a table	`/t/<tableToken>` from a printed sticker, then `/s/<publicToken>`	Read the menu, seat themself, punch orders, watch each ticket move, raise a call (water, cutlery, bill), pay a UPI charge, see the GST bill
Guest with the installed app	`/eat/<slug>`, `/track/<token>`	Pickup and delivery orders, live ticket tracking, checkout
Waiter	`/floor` — floor map, new order, table ticket	Seat and open tables, punch orders for any table, run food, ask the counter for a bill, clear a table, answer "where is my order"
Kitchen	`/kds` — four-column pass	Accept, start, mark ready — per station, nothing else
Cashier / counter	`/pos` — bills and the request queue	Issue bills, take cash and online payments, settle and refund, confirm a table clear
Owner / manager	`/app` — dashboard, orders, tables, menu, bookings, inventory, staff, reports, delivery, audit, settings	Everything operational; only the owner writes restaurant settings and integrations

One service period, end to end

1. A guest scans the sticker on table `M3`. `GET /api/public/menu/:tableToken` returns the menu plus a session for the table; the guest orders two dishes. `POST /api/public/session/:token/orders` creates an `Order` with `source: DINE_IN_QR`.
2. The order row is priced on the server (`pricing.service.ts`), saved, and broadcast: `order.created` to `kitchen:<rid>` and `restaurant:<rid>`, a `Notification` to the waiter room, and the table flips to `ORDERING`.
3. The kitchen taps Accept, then Start, then Ready on `/kds`. Each is `PATCH /api/kitchen/orders/:id/status`, guarded by `kitchen:operate` and by `ORDER_TRANSITIONS`. Recipe stock is deducted once, at the first `PREPARING`.
4. Any waiter sees `FOOD_READY` on the floor map, walks it out and marks Served — `orders:status` is held by waiters, so this is allowed.
5. The guest asks for the bill. That is a `CustomerRequest` of type `BILL` with `raisedByUserId` set. The counter accepts it, which issues one `Bill` for the table's unbilled orders; the bill and its UPI QR go back to the waiter who asked, not to whoever happens to hold the table.
6. Cash is tendered at the table, so the waiter raises `CASH_PAYMENT`; the counter records the payment, and only then may the table be cleared. `TABLE_CLEAR` exists so the floor cannot silently free a seat with money still on it.

> **Why the whole thing is realtime.** Every one of those steps is somebody else's screen. A toast that never appears is a dish that never leaves the pass, so notifications and socket invalidations are treated as function, not polish.

03 Languages, runtimes and libraries

TypeScript everywhere, one language from the browser to the database query. Node.js 20+ on the server (`engines.node: ">= 20"`), ES modules in both packages (`"type": "module"`).

Backend — `backend/package.json`

Dependency	What it is for here
`express` 4	HTTP app; routers per feature, mounted under `/api`
`mongoose`	ODM for MongoDB; schemas in `src/models` , `lean()` reads everywhere in handlers
`socket.io`	Server push; rooms per restaurant, per role, per guest token (`src/realtime/`)
`jsonwebtoken`	Two token types: a 15-minute access JWT and a 7-day refresh JWT whose value is rotated per device
`bcryptjs`	Password hashes, refresh-token hashes (so a leaked array is unusable) and one-time login codes
`zod`	Two jobs: validating the environment once at boot (`config/env.ts`) and validating every request body (`middleware/validate.ts`)
`cors` , `helmet` , `express-rate-limit` , `morgan` , `cookie-parser`	Origin allowlist, security headers, throttles on public and auth routes, request log, cookie parsing
`multer`	Menu photo uploads to disk (`middleware/upload.ts`)
`qrcode`	PNG data-URLs for table codes, the app code, the staff code, a bill's pay charge
`dotenv`	Loads `backend/.env` before the schema is parsed

Dev only: `typescript` , `tsx` (dev server and seed), `tsup` (single-file bundle), `vitest` (tests), `mongodb-memory-server` (embedded database), `@types/*` . `@clerk/express` is declared but imported nowhere — see §19.

Frontend — `frontend/package.json`

Dependency	What it is for here
`react` / `react-dom` 19	UI runtime; `useSyncExternalStore` is how socket generations reach components
`react-router-dom` 7	One `BrowserRouter` in `main.tsx` ; all routes declared in `src/App.tsx`
`zustand` 5	Three stores: `auth` (persisted), `notifications` , `toasts`
`axios`	One instance in `src/lib/api.ts` with the bearer interceptor and the single-flight refresh
`socket.io-client`	Singleton socket in `src/lib/socket.ts` with room replay on reconnect
`jsqr`	Decodes a QR through the camera in `src/lib/scan.ts` — staff never need a scanner app
`lucide-react`	Icon set

Dependency	What it is for here
`tailwindcss` 4 + `@tailwindcss/vite`	Styling. CSS-first config: there is no `tailwind.config.js`; tokens live in `src/index.css` under `@theme`

Dev only: `vite` 8, `@vitejs/plugin-react`, `typescript`. Scripts: `dev`, `build` (`tsc -b && vite build`), `preview`, `typecheck`.

Why these choices, in one line each

- **Express + Mongoose over a framework.** The API is 18 small routers; a framework would buy structure this codebase already gets from `src/modules/<feature>/`.
- **MongoDB with an embedded server.** A demo that boots with `npm run dev` and no account is worth more than a correct one that needs Atlas first. `MONGODB_URI` switches to a real cluster when it is set.
- **A hand-rolled query cache** (`src/lib/query.ts`) instead of a data library: the cache is keyed by URL and invalidated by socket events, which is the whole requirement.
- **Hand-drawn SVG charts** (`src/components/charts.tsx`) instead of a charting dependency.

04 System design

Topology as deployed

```
phone / laptop browser
         |
         |  https://smart.restaurant.aniruddhasali.in      <- one address for pages AND data
         |
  +------v-----------------------------------------------------+
  |  Vercel (project "ai", root frontend/)                      |
  |   static: /assets, /icons, /manifest, /sw.js, index+staff |
  |   rewrites (before the SPA fallback):                       |
  |     /api/*        -> https://ai-1-hsus.onrender.com/api/* |
  |     /uploads/*    -> .../uploads/*                          |
  |     /socket.io/*  -> .../socket.io/*                        |
  |     /(.*)         -> /index.html                            |
  +------+-----------------------------------------------------+
         |  plain HTTPS + long-polling
  +------v-----------------------------------------------------+
  |  Render web service "sizzle"  (rootDir backend)             |
  |   node dist/server.js  -> Express /api, Socket.IO, /uploads
  |   build: npm install && npm run build                       |
  |          (tsup bundle + frontend/dist via build-client)     |
  |   embedded MongoDB in-container: .data/mongo                |
  |   AUTO_SEED loads Saffron & Smoke into an empty database    |
  +-------------------------------------------------------------+
```

Because the browser reaches both halves from the address it is standing on, the client is built with `VITE_API_URL=` empty: `API_ROOT` is `''`, calls go to relative `/api`, and no CORS hop exists for its own traffic. The old host `ai-ecru-kappa-14.vercel.app` is the same Vercel project, so it gets the same rewrites and keeps working — that matters because stickers were printed against it.

> **One caveat worth knowing.** Vercel cannot upgrade a WebSocket through a rewrite, so Socket.IO settles on HTTP long polling on that domain. The client asks for `['websocket','polling']`, so a direct Render or laptop connection still upgrades. Nothing in the code depends on the transport.

The life of a request

1. `src/server.ts` creates the app, connects Mongo, starts the scheduler and attaches Socket.IO to the HTTP server.
2. `createApp()` in `src/app.ts`: webhooks are mounted *before* `express.json()` because a signature must be verified against the raw body; then `helmet`, `cors`, morgan, JSON, the `/uploads` static mount, `Cache-Control: no-store` on `/api`, the routers, then the SPA fallback and the error handler last.
3. `requireAuth` (`middleware/auth.ts`) verifies the access JWT, re-reads the user row and puts `{userId, restaurantId, role, name}` on `req.auth`. Scope always comes from the database, never from the token's claims alone.
4. `requirePermission('…')` or `requireRole(…)` (`middleware/rbac.ts`) resolves the role against `ROLE_PERMISSIONS` at request time, so editing the grid needs no reseed.
5. `validate({ body })` parses a Zod schema; anything unknown is rejected before a handler runs.
6. The handler is wrapped in `asyncHandler`, so a rejected promise becomes `ApiError` and the error middleware answers `{ error: { code, message } }` with the right status.

7. Business rules live in `src/services/*`, not in the handler. A write that matters also emits: `emit.ts` pushes to rooms and `notification.service.ts` persists a `Notification` row so a screen that was closed still shows it later.

How a change reaches every other screen

```
service writes Mongo  ->  emit.toKitchen(rid, 'order.ready', payload)
                      ->  notify({ recipientId })  ==  Notification row + 'notification:new' to user:<uid>
browser: socket.ts receives -> useRealtimeShell maps the event to invalidate('floor'), invalidate('kds')
       -> lib/query.ts bumps a generation -> useQuery refetches -> component re-renders
       -> toast in Toaster + bell badge for the human
```

That indirection is deliberate: a socket event says “something changed”, the data itself still comes from the same REST endpoint the screen loaded at boot. A missed event costs a stale row until the next 20-second refresh window; it cannot corrupt anything.

05 Repository layout

```
ai/
├── backend/                       the API, and the deployable that also serves the client
│   ├── src/
│   │   ├── server.ts              boot: connect db, start scheduler, attach socket.io, listen
│   │   ├── app.ts                 express app, mount order, SPA fallback, error handler
│   │   ├── routes.ts              every router's mount prefix + the public surface
│   │   ├── seed.ts                the demo tenant: staff, menu, tables, 14 days of history
│   │   ├── config/                env.ts (zod env, origins, public base), db.ts (mongoose + embedded mongo)
│   │   ├── middleware/            auth, rbac, validate, rateLimit, upload, error
│   │   ├── models/                19 mongoose schemas + index.ts barrel + phoneMatch.ts
│   │   ├── modules/<feature>/     one folder per bounded area; *.routes.ts is the HTTP face
│   │   ├── services/              business rules: order, table, billing, pricing, upi, booking,
│   │   │                          inventory, hiring, notification, report, audit, customerApp,
│   │   │                          scheduler, mockCapture
│   │   ├── realtime/              socket.ts (rooms, handshake), emit.ts (event names, helpers)
│   │   ├── integrations/          payments.ts (mock + Razorpay), delivery.ts (Swiggy/Zomato/Website)
│   │   ├── types/                 constants.ts (enums, ORDER_TRANSITIONS, ROLE_PERMISSIONS), express.d.ts
│   │   └── utils/                 asyncHandler, httpError, money, phone, query, tokens
│   ├── scripts/build-client.mjs   npm install && build in ../frontend
│   ├── tests/                     9 vitest suites + setup.ts + helpers.ts
│   ├── .data/mongo                the embedded database's files (gitignored, survives restarts)
│   └── uploads/                   multer target for menu photos (gitignored)
├── frontend/                      the browser app; two HTML entries
│   ├── index.html                 guest app  — "Sizzle"
│   ├── staff.html                 staff app  — "Sizzle Staff"
│   ├── src/
│   │   ├── App.tsx                every route, every guard
│   │   ├── main.tsx               ReactDOM root + service-worker registration
│   │   ├── index.css              Tailwind v4 @theme tokens, shared classes, keyframes
│   │   ├── layouts/               AppShell, StaffLayout, FloorLayout, KdsLayout, PosLayout
│   │   ├── pages/                 app/ guest/ kitchen/ waiter/ pos/ owner/ + auth pages
│   │   ├── components/            ui.tsx primitives, DataTable, QrScanner, UpiQr, Toaster,
│   │   │                          NotificationBell, OperationsAppModal, charts, modals
│   │   ├── hooks/                 useRealtimeShell, useInstallPrompt, useTitle
│   │   ├── lib/                   api, socket, query, types, format, statusMaps, scan, jobs,
│   │   │                          tracked, downloadQr, demoAccounts, installShortcuts
│   │   └── store/                 auth (persisted), notifications, toasts
│   ├── public/                    manifest.webmanifest, manifest-staff.webmanifest, sw.js, icons/
│   ├── scripts/make-icons.mjs     regenerates the PNG favicon set and maskable icons
│   ├── vercel.json                the four rewrites (§04)
│   └── .env / .env.production     dev needs nothing; production leaves VITE_API_URL empty
├── render.yaml                    Render Blueprint: one web service, rootDir backend
├── README.md                      run, LAN demo from a phone, env, deploy
├── DEMO-CREDENTIALS.md             every seeded sign-in and the master password
└── .gitignore                     .env, .data, uploads, dist, and the scratch files used while testing
```

Two habits are visible in the tree and worth keeping: a feature lives in **one** folder under `modules/` even when its rules reach into `services/`, and the client mirrors that split under `pages/` by persona rather than by URL.

06 Backend anatomy

Boot sequence (`src/server.ts`)

1. Parse and validate the environment. A bad variable stops the process with the field errors printed.
2. In production, refuse to start on a missing `MONGODB_URI` or a development-default token secret.
3. Connect Mongoose. With no URI, start `mongodb-memory-server` on `.data/mongo` (tests use `.data/mongo-test`) and connect to it. Only one process may hold that directory — see §19.
4. If `AUTO_SEED` is not `false` and there are zero restaurants, load the demo tenant.
5. Bind HTTP, mount Socket.IO on the same server, start the scheduler.

Modules under `src/modules/`

Folder	Mounted at	Responsibility
`auth`	`/api/auth`	Register, login, OTP request/verify, refresh, logout, `GET /me` , change password. Owns `otp.service.ts` .
`public`	`/api/public`	The unauthenticated guest surface, rate-limited separately: menu by table token, session, bill, bookings availability, hiring application, the installed-app endpoints under `/apps/…` .
`restaurants`	`/api/restaurants`	Own tenant record, the counter `/app-qr` and the `/staff-qr` codes, settings patch.
`tables`	`/api/tables`	Floor plan CRUD, status, QR generation and regeneration, waiter assignment, `/my-map` .
`orders`	`/api/orders`	Staff-side ordering, timeline, status transitions.
`kitchen`	`/api/kitchen`	The pass: board, per-station filter, stats, status changes.
`waiters`	`/api/waiters`	Floor operations: claim/seat/clear/transfer, the customer-request queue with accept, collect and settle, shift summary, live orders.
`billing`	`/api/billing`	Issue a bill, read it, cash and online settlement, print count.
`payments`	`/api/payments`	Payment records, refund, and the mock `simulate` capture; webhooks in `webhook.routes.ts` .
`menu` , `uploads`	`/api/menu` , `/api/uploads`	Categories, dishes, add-ons, prices, photos.
`inventory`	`/api/inventory`	Stock items, low-stock list, adjustments, transaction ledger.
`bookings`	`/api/bookings`	Reservations, availability computation, status lifecycle.
`staff`	`/api/staff`	Team CRUD, password reset, hiring applications and approval.
`notifications`	`/api/notifications`	The bell: list, mark read, mark all, delete.
`reports`	`/api/reports`	Dashboard, sales, product mix, payments, operations.
`audit`	`/api/audit`	Read-only audit trail.

Folder	Mounted at	Responsibility
`integrations`	`/api/integrations`	Delivery partner adapters, push an order out, pull partner orders, simulate a partner; webhook receiver for delivery.

Middleware, in the order it matters

- `auth.ts` — bearer only; 401 messages are explicit (“Please sign in to continue”, “Session expired, please sign in again”).
- `rbac.ts` — `requirePermission`, `requireRole`; a mismatch is 403 with the permission named.
- `validate.ts` — Zod schema against `body` / `query` / `params`; the parsed value replaces the raw one.
- `rateLimit.ts` — separate limiters for auth attempts, general public traffic and public writes.
- `upload.ts` — multer disk storage, jpeg/png/webp/gif, 5 MB cap, filename `<timestamp>-<token6><ext>`.
- `error.ts` — `ApiError` to status + `{error:{code,message}}`; anything unexpected logs and answers 500 without leaking internals.

07 HTTP surface

All paths below are relative to `/api`. "Auth" means a valid access token; the permission in the last column is what `requirePermission` checks.

Public — no token, rate-limited

Method	Path	Purpose
GET	`/health`	Liveness; warms a sleeping service
GET	`/public/menu/:tableToken`	Menu + table for a scanned sticker
GET	`/public/session/:publicToken`	A guest's running session
POST	`/public/table-sessions`, `/public/apps/:slug/sit`	Seat a guest at a free table
POST	`/public/session/:publicToken/orders`, `…/requests`	Order from a table; raise a call
GET	`/public/bill/:billToken`	The guest's bill and its UPI charge
GET	`/public/restaurant/:idOrSlug`, `/public/apps/restaurants`, `/public/apps/menu/:slug`, `/public/apps/orders/:trackingToken`	The installed guest app's reads
POST	`/public/apps/orders/:slug`, `…/checkout`, `…/checkout/confirm`	Pickup/delivery order and checkout
GET	`/public/bookings/availability`, `/public/hiring/:slug`	Slots for a party; open roles on the apply page
POST	`/public/bookings`, `/public/applications/:slug`	Book a table; apply for a floor job
POST	`/payments/webhook/:provider`, `/integrations/webhook/:provider`	Signature-verified; mounted before JSON parsing

Authenticated

Method	Path	Guard / note
POST	`/auth/register`, `/auth/login`, `/auth/otp/request`, `/auth/otp/verify`, `/auth/refresh`	Limited by the auth rate limiter; no token needed
GET	`/auth/me` · POST `/auth/logout`, `/auth/change-password`	Auth; logout accepts an optional `refreshToken` to end one device

Method	Path	Guard / note
GET/PATCH	`/restaurants`, `/restaurants/app-qr`, `/restaurants/staff-qr`	`settings:write`, `settings:write`, `settings:read` — a manager may open the staff code
GET/POST/PATCH/DELETE	`/tables`, `/tables/:id`, `/tables/:id/status`, `/tables/:id/assign-waiter`, `/tables/:id/regenerate-qr`, `/tables/:id/qr`, `/tables/my-map`	`tables:read` / `tables:write`
GET/POST/PATCH	`/orders`, `/orders/:id`, `/orders/:id/timeline`, `/orders/:id/items`, `/orders/:id/status`	`orders:read` / `orders:write` / `orders:status` — KITCHEN has none of the last two
GET/PATCH	`/kitchen/board`, `/kitchen/stats`, `/kitchen/orders`, `/kitchen/orders/:id/status`	`kitchen:operate`
GET/POST/PATCH	`/waiters/map`, `/waiters/requests`, `/waiters/requests/:id`, `…/accept`, `…/collect`, `…/settle`, `/waiters/tables/:id/claim|seat|clear|transfer`, `/waiters/me/summary`, `/waiters/live-orders`	`tables:*`, `requests:*`, `payments:write` for settle
POST/GET	`/billing`, `/billing/:id`, `/billing/:id/cash`, `/billing/:id/online`, `/billing/:id/print`	`billing:read|write`, `payments:write`
GET/POST	`/payments`, `/payments/:id`, `/payments/:id/refund`, `/payments/:id/simulate`	`payments:read`, `payments:refund`, `payments:write`
GET/POST/PATCH/DELETE	`/menu/categories…`, `/menu/products…`, `/menu/addons…`, `POST /uploads`	`menu:read` / `menu:write`
GET/POST/PATCH/DELETE	`/inventory`, `/inventory/low-stock`, `/inventory/transactions`, `/inventory/:id/adjust`	`inventory:read` / `inventory:write`
GET/POST/PATCH	`/bookings`, `/bookings/availability`, `/bookings/:id`, `/bookings/:id/status`	`bookings:read` / `bookings:write`
GET/POST/PATCH	`/staff`, `/staff/:id`, `/staff/:id/reset-password`, `/staff/applications`, `/staff/applications/:id/approve|reject`	`staff:read` / `staff:write`
GET/POST/DELETE	`/notifications`, `/notifications/read-all`, `/notifications/:id/read`, `/notifications/:id`	Any signed-in user; rows are scoped to `recipientId`
GET	`/reports/dashboard`, `/sales`, `/products`, `/payments`, `/operations`	`reports:read`
GET	`/audit`	`audit:read`

Method	Path	Guard / note
GET/POST	`/integrations`, `/integrations/orders`, `/integrations/push/:orderId`, `/integrations/simulate/:provider`	`integrations:write` (owner only)

Every authenticated read is filtered by `req.auth.restaurantId`; a token from another tenant cannot reach these rows even with the right permission, because the query always carries the restaurant id.

08 Data management

Nineteen collections in `src/models`, all tenant-scoped by `restaurantId` except `Restaurant` itself and `Counter`. Reads in handlers use `.lean()`; writes go through a document so pre-save hooks run.

Collection	Fields that matter	Indexes / notes
`Restaurant`	`slug`, owner, address, `taxPercent` (0–28, default 5), `serviceChargePercent`, hours, `payment.upiId/upiName`, branding, and a `settings` block: `acceptingOrders`, `autoAcceptOrders`, `billFooterNote`, booking window and slot sizes, `allowWaiterCash`, `deliveryEnabled`, `openRoles`	`slug` unique, lowercased
`User`	role, name, email, phone, `passwordHash` (`select:false`), status, `refreshTokens[]` (`select:false`), `lastLoginAt`	email and phone unique+sparse; `phoneDigits` index; pre-save stamps `phoneDigits`
`Table`	`number`, capacity, section, status (8 states), `qrToken`, `activeSessionId`, `assignedWaiterId`, `lastCleanedAt`	`qrToken` unique; `(restaurantId, number)` unique; `(restaurantId, status)`
`TableSession`	table, `publicToken`, guestCount, customerName, status, `openedVia` QR or STAFF, opened/closed/lastActivity	`publicToken` unique; `(restaurantId,status)`, `(tableId,status)`
`Order`	`orderNumber`, source (9 values), fulfilment, paymentMode, table/session/waiter/cashier refs, customer fields, `externalOrderId`, embedded `items[]` (product, name, price, qty, taxPercent, station, isVeg, addons, notes, lineTotal), totals block, status, `paymentStatus`, `billId`, the five timestamps, `statusHistory[]`, `trackingToken`	`(restaurantId,orderNumber)` unique; `trackingToken` unique+sparse; `(restaurantId,status,createdAt↓)`; `(restaurantId,tableSessionId)`; partial-unique `(restaurantId,source,externalOrderId)` — the idempotency key that stops a partner webhook double-firing
`Bill`	`billNumber`, `orderIds[]`, session/table, totals, `taxBreakup[]`, status, paymentStatus, `publicToken`, issuedBy, printCount	`(restaurantId,billNumber)` unique; `publicToken` unique

Collection	Fields that matter	Indexes / notes
`Payment`	bill, orders, method, provider, amount/tendered/change, provider ids and `checkoutUrl`, status, refunds[], collectedBy, verifiedAt	`createdAt↓`; `providerOrderId` sparse
`Product` / `Category` / `Addon`	price, `taxPercent` or inherit, isVeg, station, `prepMinutes`, `addonIds`, `recipe[{inventoryItemId, qty}]`, tags, sortOrder, isActive	name index; `(restaurantId,name)` unique on Addon
`InventoryItem` / `InventoryTransaction`	unit, stock, lowStockThreshold, costPerUnit, supplier; transaction type, qty, `balanceAfter`, refOrder, byUser	`(itemId, createdAt↓)` — the ledger is append-only
`CustomerRequest`	table, session, type (9 values incl. `TABLE_CLEAR`), note, status, `raisedByUserId`, handledBy, `billId`, `tendered`, collectedBy	`(status, createdAt↓)`. `raisedByUserId` is what lets a bill travel to the waiter who asked
`Booking`	table, customer, startAt, durationMinutes, guests, status (7 values), source, reminderSentAt, statusHistory	`(tableId, startAt)` for the availability overlap query
`Notification`	recipient, type, title, body, entityType/Id, `data`, readAt	`(recipientId, createdAt↓)`, `(recipientId, readAt)`
`StaffApplication`	name, phone, phoneDigits, role (floor only), note, status, decidedBy, userId	partial-unique `(restaurantId, phoneDigits, status)` while PENDING — one live application per number
`LoginCode`	user, phoneDigits, `codeHash` (`select:false`), expiresAt, attempts, consumedAt	TTL index on `expiresAt` with `expireAfterSeconds: 0` — Mongo deletes spent codes
`AuditLog`	actor, action, entity, metadata, ip	`(action, createdAt↓)`
`Counter`	`key`, `seq`	`key` unique; `nextSeq()` does an atomic `$inc` upsert — invoice and order numbers come from here, never from a client

Where the data physically lives

- **Nothing set:** `mongodb-memory-server` runs a real mongod against `backend/.data/mongo`. It survives restarts locally, which is why a reseed is a deliberate act (`npm run seed:fresh`, or deleting that folder).
- **`MONGODB_URI` set:** a real cluster (Atlas M0 in the README). Then `npm run seed` also works from a Render shell, because both containers see the same database.
- **Uploaded photos:** `backend/uploads` on the container's disk. A redeploy wipes them — object storage is the fix for a real install; committed art under `frontend/public` survives.
- **Every token and QR in the database** is regenerated by a reseed. Codes printed before a rebuild point at tokens that no longer exist.

09 State machines

Transitions are declared once, in `src/types/constants.ts`, and enforced in the service — a client cannot jump a row by sending a status it likes.

Order

```
PLACED ──> ACCEPTED ──> PREPARING ──> READY ──> SERVED ──> COMPLETED
   |            |             |           |
   └────────────┴─────────────┴───────────┴──> CANCELLED      READY may also go straight to COMPLETED
```

Target status	Roles allowed to move an order into it (`TRANSITION_ROLES`)
`ACCEPTED`, `PREPARING`, `READY`	KITCHEN, MANAGER, OWNER
`SERVED`	adds WAITER and CASHIER
`COMPLETED`	CASHIER, WAITER, MANAGER, OWNER
`CANCELLED`	CASHIER, MANAGER, OWNER

A settlement is a system transition: `transitionOrder(…, { system: true })` skips the role check and may complete an order from any reachable state, which is how paying a bill finishes the tickets behind it.

Table, session, booking

- **Table** `AVAILABLE → RESERVED → OCCUPIED → ORDERING → FOOD_READY → BILL_REQUESTED → PAYMENT_PENDING → CLEANING → AVAILABLE`. A clear that still has money on it stops at `PAYMENT_PENDING`.
- **Session** `OPEN → BILL_REQUESTED → PAYMENT_PENDING → CLOSED`. The session id is the key the punch queue uses (§10).
- **Booking** `PENDING → CONFIRMED → ARRIVED → SEATED → COMPLETED`, with `CANCELLED` and `NO_SHOW` reachable from the first three.

Side effects that are part of the machine

- First move to `PREPARING` deducts the recipe once, and writes `InventoryTransaction` rows.
- `READY` notifies the waiter who holds the table *and* the waiter room, so a colleague can run it.
- Every transition appends to `statusHistory[]`, which is what `/orders/:id/timeline` renders.
- Stock and status changes are audited through `audit.service.ts`.

10 The two floor rules

Both of these came from real failures on the floor and both are covered by tests in `backend/tests/cashflow.test.ts`. Do not "simplify" them away.

Rule 1 — one fire per table

A punch that arrives while the table's current order is still `PLACED` or `ACCEPTED` — that is, before the kitchen has begun cooking — is **merged into that order** instead of creating a second ticket. The response carries `{ merged: true }` so the caller can say "added to the running order" rather than showing a new ticket number. Once the order is `PREPARING` or later, the next punch is a genuine second round with its own ticket.

```
order.service.ts
  createOrder(input)                  // entry point for every source
    └─ key = String(input.tableSessionId)
       tail = (punchQueues.get(key) ?? Promise.resolve()).then(() => placeOrder(input))
       punchQueues.set(key, settled)  // the next punch on this table waits for this one
       ...cleanup when settled
  placeOrder(input)                   // the merge check happens inside the queue
```

The queue exists because two phones can punch the same table within a few milliseconds of each other, and both would read "no open order" before either wrote. Serialising per session closes that window without refusing anyone. `MERGED_SOURCES` limits the merge to `DINE_IN_QR` and `DINE_IN_WAITER`: a guest tap and a waiter punch land on the same ticket, which is exactly the case that produced two bills for one dish.

> **Known limit.** `punchQueues` is an in-process `Map`. It is correct on one API instance — which is how this project deploys — and would need a database-level guard (a unique partial index on "the table's open unbilled fire") to be correct across replicas.

Rule 2 — the name on a table is a label, not a lock

Every waiter works every table. `takeTableForWaiter()` writes the seating waiter's name only while `assignedWaiterId` is still null, and the write is conditional so two phones tapping the same free table settle on one name instead of overwriting each other. Nothing downstream refuses a colleague: any waiter can add items, serve, bill and clear a table they are not named on. The floor shows "with Sneha Patil" as information, and `NewOrder` says in words that anything punched there lands on the same ticket, so the party is billed once.

Related decisions: a cleared table loses its waiter name (`tables.routes.ts` sets `assignedWaiterId = null`), and the seed leaves free tables unnamed — a demo floor with a name printed on every seat makes "take this table" impossible to reach and looks like the ghost shift is working.

Where a bill goes when it is asked for

A `BILL` or `TABLE_CLEAR` request records `raisedByUserId`. When the counter accepts it and issues the bill, the notification goes to the raiser *and* the name on the table, deduplicated; if neither resolves it goes to the waiter room. This is why a waiter who raised the call can open the bill and show the guest the QR even when the table belongs to someone else.

11 Money

Pricing is computed on the server, always

`services/pricing.service.ts` recomputes every line from the database row: unit price from `Product`, add-on prices, per-item `taxPercent` or the restaurant default, quantity, line total, then subtotal, tax, service charge, discount, and `roundOff` to the nearest rupee. A client-sent total is ignored. `tests/pricing.test.ts` exists precisely to keep that true.

Bill

- Number `INV-<YYYYMMDD>-<NNNN>` from the atomic `Counter`; one bill gathers the table's unbilled orders and stamps `billId` on each.
- `taxBreakup[]` splits CGST and SGST for a GST bill the guest can read at `/bill/<billToken>`.
- `publicToken` makes the bill readable without a login — that is the QR the waiter shows.
- `printCount` and `lastPrintedAt` exist because a printed bill is a legal artefact.

UPI and payments

- `upi.service.ts` builds `upi://pay?pa=&pn=&am=&cu=INR&tn=` from the restaurant's `payment.upiId` and renders a QR data-URL. No UPI handle configured means no charge is offered, not a broken link.
- `integrations/payments.ts` has two providers: a mock (which returns a `checkoutUrl` to `/pay/mock` and needs an explicit `POST /payments/:id/simulate` or a verified webhook to mark money in) and Razorpay wiring behind optional keys. A payment is never marked paid by a client assertion.
- Cash at a table is a two-step dance on purpose: the waiter raises `CASH_PAYMENT` with the tendered amount, the counter records it (`…/collect`, `…/settle`), and only then may the table be freed. Waiters hold `billing:write` and `payments:read` but never `payments:write`.
- Refunds are a separate permission (`payments:refund`) and a separate status, including `PARTIALLY_REFUNDED`.

12 Auth, sessions and permissions

Tokens

- **Access** JWT `{ sub, rid, role, name }`, TTL `ACCESS_TOKEN_TTL` (15 minutes). Sent as `Authorization: Bearer`. The middleware re-reads the user row, so a suspension or role change takes effect on the next request.
- **Refresh** JWT `{ sub, v }`, TTL `REFRESH_TOKEN_TTL_DAYS` (7 days). Only the bcrypt hash of `v` is stored, in `User.refreshTokens[]`.
- **Per-device sessions.** Each sign-in appends a slot; the list is capped at 5 and the oldest is evicted. Refreshing rotates only the caller's slot, so the counter tablet stays signed in while a phone refreshes. `POST /auth/logout` with a `refreshToken` ends that device; without one it ends all of them. Resetting a staff password clears the whole list.
- The client's 401 interceptor refreshes in a single flight and replays the original request once (`_retry`).

OTP sign-in

Floor roles only — `WAITER`, `KITCHEN`, `CASHIER` — because those are the accounts a hired person may not have a password for yet. Six digits, five-minute TTL, five attempts, three requests per fifteen minutes, stored as a bcrypt hash in `LoginCode` with a TTL index that garbage collects it. There is no SMS gateway, so outside production `deliver()` logs the code and returns it; that is a demo door, and it is why owner and manager are excluded.

The permission grid (`ROLE_PERMISSIONS`)

Role	Permissions
`OWNER`	All 26, including `settings:write`, `integrations:write` and `audit:read`.
`MANAGER`	Everything operational — menu, orders, kitchen, billing, payments and refunds, bookings, tables, requests, staff, inventory, reports, audit, `settings:read`. Not `settings:write`, not `integrations:write`.
`CASHIER`	`menu:read`, `orders:read`, `orders:status`, `billing:read|write`, `payments:read|write`, `tables:read`, `bookings:read`, `requests:read|write`.
`KITCHEN`	`menu:read`, `orders:read`, `kitchen:operate`. Deliberately no `orders:status` — the pass uses the kitchen routes.
`WAITER`	`menu:read`, `orders:read|write|status`, `tables:read|write`, `bookings:read|write`, `requests:read|write`, `billing:read|write`, `payments:read`.

The client keeps a mirrored copy in `frontend/src/store/auth.ts` as `PERMISSIONS` plus `can(role, permission)`, and it is used only to decide what to render. The server checks again on every route; hiding a button is not a security boundary.

Demo master password

`DEMO_MASTER_PASSWORD` (`Sizzle@Master1`) opens any active account — but only in the tenant whose slug is `saffron-and-smoke`, and only while `NODE_ENV` is not production, where the value arrives empty. A restaurant that onboarded itself through `/register` is a different tenant and the master gets nowhere near it. Suspended

accounts are refused before the master is compared, and a failure caused by the master is indistinguishable in wording from any wrong password.

13 Realtime contract

Socket.IO on `/socket.io`, same HTTP server. The handshake takes `auth.token`; a valid JWT joins the rooms its role implies, and an invalid or absent token is not rejected — it becomes a guest socket that may only join by public token.

Room	Who is in it
`restaurant:<rid>`	Every staff socket of that tenant
`kitchen:<rid>`, `cashier:<rid>`	By role, for the pass and the counter
`waiter:<uid>`, `user:<uid>`	Named delivery to one person's devices
`session:<publicToken>`, `order:<trackingToken>`	A guest's own table or ticket, joined only after the server validates the token

Client → server: `join:session`, `join:order`, `leave:session`, `leave:order`, each with an `{ ok }` acknowledgement. The client keeps its own set of joined tokens and replays them on every `connect`, because a reconnect silently loses room membership.

Event	Raised when
`connection:ready`	Handshake accepted — the client's "live" chip keys off this
`order.created|accepted|preparing|ready|served|completed|cancelled|updated`	Any order write; `created` reaches kitchen, floor and counter rooms
`customer.requested`, `request.updated`	A guest or waiter raises a call, and when it is handled
`bill.requested|created|updated|paid`	The bill lifecycle
`payment.succeeded|failed|refunded`	A capture, a failed webhook, a refund
`booking.created|updated|reminder`	A booking, or the scheduler's reminder
`table.updated|cleaned|available`, `session.updated`	Floor state
`menu.updated`, `staff.updated`	Anything a cached read would go stale on
`inventory.updated`, `inventory.low_stock`	Stock moves, and the threshold crossing

Event	Raised when
`notification:new`	A persisted `Notification` row, so the bell and the toast agree
`delivery.order`	A partner webhook landed

Names are declared once in `src/realtime/emit.ts` as `Events`; a handler that emits a string literal is a bug waiting to drift.

14 Frontend anatomy

Two entries, one bundle graph

`vite.config.ts` declares two rollup inputs: `index.html` ("Sizzle", the guest app, `manifest.webmanifest`) and `staff.html` ("Sizzle Staff", `manifest-staff.webmanifest`). Both load the same `main.tsx`; the manifest and the home-screen name are the only difference, and that is what lets a waiter install the floor and a guest install ordering on the same phone as separate apps.

Routes and guards (`src/App.tsx`)

Path	Screen	Guard
`/`	Hub	none — the chooser for a phone on the Wi-Fi
`/login`, `/register`, `/staff.html`	Login, Register, StaffInstall	none
`/eat`, `/eat/:slug`, `/track/:token`	AppLanding, CustomerApp, AppTrack	none
`/t/:tableToken`, `/s/:publicToken`, `/bill/:billToken`	GuestMenu, GuestSession, GuestBill	the token is the credential
`/book/:slug`, `/apply/:slug`	PublicBooking, ApplyJob	none
`/app/*`	StaffLayout: dashboard, orders, order detail, requests, tables, menu, bookings, inventory, staff, reports, delivery, audit, settings	signed in and `orders:read`; nav items filtered per permission
`/floor/*`	FloorLayout: WaiterFloor, FloorMap, NewOrder, BookingsPage	`tables:read`
`/kds`	KdsBoard	KITCHEN, MANAGER or OWNER
`/pos/*`	PosLayout: BillsPage, PosRequests	`billing:read`
`*`	NotFound	—

Data layer

- `lib/api.ts` — `API_ROOT` from `VITE_API_URL`, empty meaning same host. Attaches the bearer token unless a caller already set one, refreshes once on a 401 for non-auth routes, and normalises errors through `errMsg()` which reads `data.error.message`. `mediaUrl()` turns a stored `/uploads/…` path into something loadable from another origin.
- `lib/query.ts` — a Map keyed by a string, `peek / write / patch / invalidate / invalidateMany`, and `useQuery(key, url)` with a 20-second freshness window plus a generation counter so an invalidation from a

socket refetches the mounted screen. `patch` is what makes an optimistic update possible without a library.

- `lib/socket.ts` — the singleton socket, a `generation` number that is bumped whenever the socket object is replaced (subscribers read it through `useSyncExternalStore`, so a token refresh cannot leave every component bound to a dead socket), the reconnect room replay, and the failed-handshake counter that mints a fresh access token after two consecutive failures.
- `store/auth.ts` — persisted under `sizzle.auth`: user, restaurant, both tokens, `ready`; `login`, `loginWithCode`, `register`, `logout`, `bootstrap`, and the mirrored permission map.
- `store/notifications.ts` and `store/toasts.ts` — in-memory; the bell hydrates from the API, toasts expire on a timer.
- `hooks/useRealtimeShell.ts` — mounted by all four staff layouts; maps each socket event to the query keys it invalidates and to a role-targeted toast. This is the file to edit when a new event needs to reach a screen.

UI system

Tailwind v4 with no config file: the design tokens live in `src/index.css` under `@theme` — display font **Bricolage Grotesque**, body **Inter**, and the palette scales `ink` (warm charcoal), `ember` (brand orange), `saffron`, `leaf`. Shared classes `.card`, `.field`, `.label` and the `sizzle-in`, `sizzle-pulse`, `sizzle-slide` keyframes are defined there. Primitives are in `components/ui.tsx` (Button, LinkButton, Card, Modal, Drawer, Field, Tabs, StatTile, VegDot, DishThumb, Toggle, SegmentedControl, Spinner), with `DataTable`, `QrScanner`, `UpiQr`, `Toaster`, `NotificationBell`, `ImagePicker`, `OperationsAppModal` and `charts.tsx` above them. Status-to-colour copy is centralised in `lib/statusMaps.ts` so “READY” reads the same on the pass and the floor.

Types in `lib/types.ts` are a hand-maintained mirror of the backend shapes — not generated. When a field changes on a model, both files must move together; nothing will tell you.

15 PWA, QR codes and installables

Install

- Two manifests: guest (`start_url /eat`) and staff (`start_url /staff.html` , categories business/productivity/food, shortcuts Office, Floor, Kitchen, Counter). Theme and background `#1c1917` , `display: standalone` with the newer `override` too.
- `public/sw.js` , cache `sizzle-v3` : precaches the shell (both manifests, every icon, `/` , `/eat` , `/staff.html`); GET only; **skips** `/api` , `/uploads` and `/socket.io` so data is never served from a stale cache; navigations network-first with a fallback to the cached shell; static assets cache-first.
- `hooks/useInstallPrompt.ts` handles the deferred prompt; `lib/installShortcuts.ts` writes a `.url` and a `.bat` (`msedge|chrome --app=<url>`) for a desktop that will not accept a PWA install.

Which code points where, and how the host is chosen

Code	Encodes	Host rule
Table sticker	`/t/<qrToken>`	`PUBLIC_BASE_URL` (`tables.routes.ts`) — set it to the address customers reach
Counter / bag / box	`/eat/<slug>`	`clientBase(req.headers.origin)` in `restaurant.routes.ts` : the request's own public, allowlisted origin, else `PUBLIC_BASE_URL`
Staff board	`/staff.html?r=<slug>`	same rule, so a laptop print stays on the laptop address
Bill pay code	`/bill/<publicToken>`	`PUBLIC_BASE_URL`
UPI charge	`upi://pay?…`	no host at all — a payment deep link
Anything the page itself links to	floor, session, install shortcuts	`window.location.origin` , which is why the same build works on both deployed addresses

Every code on screen can also be saved as a PNG (`lib/downloadQr.ts`), named for what it points at. Because the API returns the code as a data-URL, the saved file and the on-screen image cannot disagree — a print dialog that re-encodes could.

> **The rule in one sentence.** A printed code must encode the address a customer's phone can open, never the service that computed it: the Render free tier sleeps, and a sticker that takes thirty seconds to wake reads as a broken QR.

16 Configuration reference

Parsed once, in `backend/src/config/env.ts`, with Zod. Anything invalid stops the process.

Variable	Default	What it changes
`NODE_ENV`	`development`	Production refuses a missing `MONGODB_URI` or a placeholder secret, and switches off the demo master password and the OTP echo
`PORT`	`4000`	HTTP bind
`MONGODB_URI`	unset → embedded	Real cluster vs the in-container database
`ACCESS_TOKEN_SECRET` / `REFRESH_TOKEN_SECRET`	dev placeholders	Signing; change before deploying
`ACCESS_TOKEN_TTL` / `REFRESH_TOKEN_TTL_DAYS`	`15m` / `7`	Session lifetimes
`CORS_ORIGIN`	built-in list	Comma-separated origins checked by HTTP CORS *and* the socket handshake. The built-in list (`RENDER_EXTERNAL_URL`, both client hosts, localhost) is always added, so a stale dashboard value cannot lock a known client out
`PUBLIC_BASE_URL`	hosted → the client domain; local → `http://localhost:5173`	Host baked into table QR, guest bills, booking links and the mock checkout. A private address here is overridden with a boot warning, because no customer's phone can open a laptop IP
`DEMO_MASTER_PASSWORD`	`Sizzle@Master1`	The demo door; blank switches it off, production always is
`AUTO_SEED`	on	Set `false` to boot an untouched database clean
`UPLOAD_DIR` / `STATIC_DIR`	`uploads` / auto	Where photos go; where the built client is found (`../frontend/dist` by default)
`PAYMENT_WEBHOOK_SECRET`, `SWIGGY_/ZOMATO_/WEBSITE_WEBHOOK_SECRET`	dev values	HMAC keys every webhook body is compared against before it is trusted

Variable	Default	What it changes
`SWIGGY_API_BASE/KEY`, `ZOMATO_API_BASE/KEY`, `RAZORPAY_KEY_ID/SECRET/WEBHOOK_SECRET`	unset	Live partner and gateway calls; unset means the adapter simulates
`RENDER_EXTERNAL_URL` / `RENDER_EXTERNAL_HOSTNAME`	set by Render	How the API knows it is deployed — used instead of `NODE_ENV`, because Render leaves the latter at its default

Frontend variables: `VITE_API_URL` only, read in `lib/api.ts` and again in `lib/socket.ts` (that duplication is deliberate — the socket module must be importable *by* the HTTP layer). Never put a secret in a `VITE_*` variable: Vite inlines it into the bundle the browser downloads.

17 Build, test and deploy

Running it locally

```
cd backend  && npm install && npm run dev     # API on :4000, embedded Mongo under .data/mongo
cd frontend && npm install && npm run dev     # Vite on :5173, proxying /api /uploads /socket.io
npm run seed:fresh                            # rebuild the demo tenant (new tokens, new QR codes)
```

To demo on a phone on the same Wi-Fi: Vite binds every interface and prints the LAN address; put that address in `backend/.env` as `PUBLIC_BASE_URL` and add it to `CORS_ORIGIN`, then restart the API. Omitting the LAN origin is the usual reason a phone loads the page but never gets live updates. If the phone spins and never connects, that is router client isolation, not this repository.

Gates that must be green before a push

```
backend :  npm run typecheck      # tsc --noEmit  - the build does NOT typecheck, this does
backend :  npm run test           # vitest run    - 9 files, embedded Mongo on .data/mongo-test
backend :  npm run build          # tsup bundle + scripts/build-client.mjs (builds ../frontend)
frontend:  npm run typecheck
then:      live smoke on the deployed URLs, and a secret read of the diff about to go out
```

What each suite protects

File	Behaviour groups
`cashflow.test.ts`	One bill per table, cash at the table, **one fire per table**, **every waiter works every table** — the two floor rules above
`orders.test.ts`	Totals computed from the menu and never from the client; the order machine walked to completion with stock deducted once
`pricing.test.ts`	Money helpers, line totals, discounts, `computeTotals`
`auth.test.ts`	Sign-in and the master-password rules; one account on two devices, including the oldest-device eviction
`customerApp.test.ts`	The guest ordering app and seating from it
`bookings.test.ts`	Availability, the smallest table that fits a party, the booking lifecycle
`upi.test.ts`	The scan-to-pay charge carries the exact amount and the bill reference
`hiring.test.ts`	Applying from a sticker, approving, OTP codes, which roles are open
`webhooks.test.ts`	Payment gateway and delivery partner webhook handling

Deploy

1. **Render** — `render.yaml` is a Blueprint (*New → Blueprint* at this repository). Root directory `backend`, build `npm install && npm run build`, start `node dist/server.js`, health check `/api/health`. Only `MONGODB_URI` is left to fill in.
2. **Vercel** — project root `frontend`, so `frontend/vercel.json` is read. The four rewrites in §04 must stay ahead of the SPA fallback; a catch-all first answers an API call with `index.html`, and an unknown top-level key in that

file fails the build outright.

3. **Push to `main`** deploys both. Expect Render to take a few minutes and to wipe and reseed the embedded database on the way up.

> **Two deployment facts that surprise people.** A redeploy resets the demo tenant, so every QR token, session token and JWT changes — codes printed from the previous instance stop resolving. And the service's `NODE_ENV` is not `production` on Render even though `render.yaml` sets it, so anything that must apply to the deployment has to key on `RENDER_EXTERNAL_URL` being present, not on `env.isProd`.

18 Runbook — symptom to file

Symptom	Look here	Why
Page loads, no live updates	`config/env.ts` origins, `hooks/useRealtimeShell.ts`	The origin is not on the allowlist, or the event has no invalidation mapping
Every API call returns HTML	`frontend/vercel.json` order	The SPA catch-all is ahead of the `/api` rewrite
Vercel build fails on a config file	`frontend/vercel.json`	Unknown top-level keys are rejected; comments belong in the README
A printed QR opens an error page	`PUBLIC_BASE_URL`, `clientBase()`, whether a reseed happened since	Wrong host baked in, or the token no longer exists
Sign-in fails with no clear reason	`modules/auth/auth.service.ts`, `middleware/auth.ts`	Suspended status, an evicted refresh slot, or a tenant the master password does not reach
Waiter cannot change a status	`types/constants.ts` `ROLE_PERMISSIONS`, `TRANSITION_ROLES`	KITCHEN has no `orders:status`; only listed roles may enter a status
Table refuses to clear	`services/billing.service.ts`, `modules/waiters`	Money is still open — a payment must be recorded at the counter first
Duplicate ticket for one dish	`services/order.service.ts` `punchQueues`, `MERGED_SOURCES`	Rule 1; check whether the source is outside the merge set
Menu photo gone after a deploy	`UPLOAD_DIR` on container disk	Expected; only committed files under `frontend/public` survive
First request after a quiet period takes ~30s	Render free tier	`/api/health` warms it; a real install needs a paid instance
Two dev servers fight over the database	`config/db.ts`, `.data/mongo`	Only one mongod may hold the lock directory

Diagnosing the live pair without a browser

```
curl -s https://smart.restaurant.aniruddhasali.in/api/health      # JSON = proxy works; HTML = rewrite missing
curl -s -o /dev/null -w "%{http_code} %{content_type}\n" \
     https://smart.restaurant.aniruddhasali.in/socket.io/?EIO=4&transport=polling
curl -s -D - -o /dev/null -X OPTIONS https://ai-1-hsus.onrender.com/api/auth/login \
     -H "Origin: https://smart.restaurant.aniruddhasali.in" \
     -H "Access-Control-Request-Method: POST" | grep -i access-control-allow-origin
```

19 Deliberate mocks, limits and debts

Mocked on purpose

- **Payments.** A card or UPI settlement needs the explicit `POST /payments/:id/simulate` the counter makes, or a signed webhook. Money is only ever marked in by an event, never by a client saying so.
- **Delivery partners.** Swiggy, Zomato and the website channel run through adapters that simulate unless API keys are present; `POST /integrations/simulate/:provider` exists so the floor can be rehearsed.
- **SMS.** No gateway, so the OTP code is logged and returned outside production — which is why floor roles only may use it.

Structural limits

- **Single process.** The punch queue and the scheduler's interval are in-memory. Correct on one instance; a second replica needs database-level guards and a distributed lock.
- **Embedded database on the free tier.** Every deploy wipes and reseeds. Real use means `MONGODB_URI` plus object storage for uploads.
- **Types are mirrored by hand** between `backend/src/models` and `frontend/src/lib/types.ts`, and permissions are duplicated between `constants.ts` and `store/auth.ts`. Nothing enforces they agree.
- **Rate limits are in-memory**, so they reset with the process and do not span replicas.
- **WebSocket upgrade is lost behind the Vercel rewrite**; realtime on that domain rides long polling.

Debts worth scheduling

- `@clerk/express`, `@clerk/clerk-react` and `@clerk/react` are declared in both `package.json` files and imported nowhere; the two files left behind (`backend/src/modules/auth/clerk.webhook.ts`, `frontend/src/components/ClerkAuthBridge.tsx`) are one-line stubs saying "no longer used". Auth is the custom JWT scheme in §12. Remove all of it.
- `onRealtimeAuthFailure()` is exported from `lib/socket.ts` but no module calls it, so the socket's self-healing token refresh never fires — the HTTP layer's own 401 path is what keeps sessions alive today. Wire it or delete it.
- Realtime events are untyped on the client: the names in `emit.ts` and the strings in `useRealtimeShell.ts` agree only by discipline.
- Menu photos have no checksum or dedupe; re-uploading the same dish art accumulates files.

20 Glossary

Term	Meaning in this codebase
Fire / ticket	One `Order` for one table session — the kitchen's unit of work
Punch	Add dishes to a table, from the guest's phone or a waiter's
Pass	The kitchen board at `/kds`, four columns by status
Floor	The waiter view at `/floor`: tables, seats, calls
Counter	The POS at `/pos`: bills, payments, the request queue
Session	`TableSession` — one sitting at one table, with a public token a guest can hold
Claim	Put your name on a free table. A label; never a lock
Clear	Free a table for the next party, allowed only once money is settled
Tenant	One `Restaurant` row and everything scoped to its id
Open roles	`FLOOR_ROLES` — the jobs a stranger may apply for from a sticker
Invalidation	Bumping a query key so mounted screens refetch

Written against commit `bbfa7bb` on `main`. Where this document and the code disagree, the code is right and this page is a bug report.